

Teoria da Computação

Decidibilidade: problemas decidíveis, reconhecíveis e indecidíveis

Prof. Me. João Paulo Biazotto

Universidade Estadual do Paraná – Campus Apucarana
Bacharelado em Ciência da Computação

Teoria da Computação
2026

Siglas e notações da aula

Antes de discutir decidibilidade, vamos padronizar as siglas usadas nos slides. Isso evita que a notação apareça antes de ser definida.

Sigla	Significado
AFD	Autômato Finito Determinístico
AFN	Autômato Finito Não Determinístico
ER	Expressão Regular
GLC	Gramática Livre de Contexto
MT	Máquina de Turing

Síntese

O capítulo usa subscritos em inglês, como *DFA* e *NFA*. Em português, leremos *DFA* como AFD e *NFA* como AFN.

Notações para problemas de decisão

As notações abaixo representam linguagens. Cada linguagem contém as codificações das instâncias cuja resposta ao problema é SIM.

Notação	Leitura intuitiva
A_{DFA}	Aceitação para AFDs: dado $\langle B, w \rangle$, o AFD B aceita w ?
A_{NFA}	Aceitação para AFNs: dado $\langle B, w \rangle$, o AFN B aceita w ?
A_{REG}	Aceitação/geração para expressões regulares: uma ER gera w ?
E_{DFA}	Vazio para AFDs: dado $\langle A \rangle$, temos $L(A) = \emptyset$?
EQ_{DFA}	Equivalência para AFDs: dois AFDs reconhecem a mesma linguagem?

Como ler A_{DFA} , A_{NFA} e E_{DFA}

A letra principal indica o tipo de problema:

- A vem de *acceptance*: problema de aceitação.
- E vem de *emptiness*: problema de linguagem vazia.
- EQ vem de *equivalence*: problema de equivalência.

O subscrito indica o formalismo envolvido:

- DFA : *deterministic finite automaton*, isto é, AFD.
- NFA : *nondeterministic finite automaton*, isto é, AFN.
- REX : expressão regular.

Definições antecipadas das principais linguagens

Durante a aula, usaremos principalmente estas linguagens:

$$A_{DFA} = \{\langle B, w \rangle \mid B \text{ é um AFD que aceita } w\}$$

$$A_{NFA} = \{\langle B, w \rangle \mid B \text{ é um AFN que aceita } w\}$$

$$E_{DFA} = \{\langle A \rangle \mid A \text{ é um AFD e } L(A) = \emptyset\}$$

Síntese

A partir deste ponto, quando aparecer A_{DFA} , A_{NFA} ou E_{DFA} , a sigla já terá sido definida.

Objetivos da aula

Ao final desta aula, espera-se que vocês consigam explicar, com exemplos, os principais conceitos do capítulo de decidibilidade.

- Entender por que problemas computacionais podem ser representados como linguagens.
- Diferenciar uma linguagem **decidível** de uma linguagem apenas **Turing-reconhecível**.
- Descrever algoritmos decisores para problemas envolvendo AFDs, AFNs, expressões regulares e GLCs.
- Compreender por que algumas estratégias ingênuas não são decisores, mesmo quando parecem algoritmos razoáveis.
- Entender a ideia geral da diagonalização e sua relação com problemas indecidíveis.
- Explicar por que A_{TM} é reconhecível, mas não decidível.
- Usar a relação entre reconhecibilidade e co-reconhecibilidade para interpretar $\overline{A_{TM}}$.

Estrutura sugerida da aula

A aula foi planejada para aproximadamente **50 minutos de explicação**, com exemplos intuitivos ao longo da apresentação, seguida de **3 exercícios guiados** para resolver com os alunos.

- 1 Motivação: o que significa dizer que um problema não tem solução algorítmica?
- 2 Problemas como linguagens: codificação, entrada e aceitação.
- 3 Problemas decidíveis sobre linguagens regulares.
- 4 Problemas decidíveis sobre linguagens livres de contexto.
- 5 Reconhecibilidade e a diferença entre reconhecer e decidir.
- 6 Diagonalização, A_{TM} e indecidibilidade.
- 7 Co-reconhecibilidade e síntese final.
- 8 Exercícios guiados com solução passo a passo.

A pergunta central

A computação normalmente é apresentada como uma área voltada a construir soluções: algoritmos de ordenação, busca, compiladores, bancos de dados, sistemas operacionais, inteligência artificial etc.

Neste capítulo, a pergunta muda de foco:

Existem problemas que nenhum algoritmo consegue resolver?

A resposta é sim. O objetivo da aula é entender o significado dessa afirmação. Não estamos dizendo que ainda não encontramos um algoritmo. Estamos dizendo que, para certos problemas, é possível provar que um algoritmo geral não pode existir.

Por que estudar problemas insolúveis?

Estudar problemas insolúveis pode parecer estranho: se um problema não pode ser resolvido, por que gastar tempo com ele?

- Primeiro, porque isso ajuda a entender os limites reais dos computadores. Um computador não é uma ferramenta mágica; ele possui limitações matemáticas.
- Segundo, porque evita que procuremos uma solução geral para um problema que precisa ser reformulado.
- Terceiro, porque muitos problemas práticos de verificação automática têm relação com esses limites.

Exemplo

Uma ferramenta poderia verificar automaticamente se qualquer programa está correto em relação a qualquer especificação? A teoria mostra que essa ambição, em sua forma mais geral, é impossível.

O papel da Máquina de Turing

No capítulo anterior, a Máquina de Turing foi apresentada como modelo geral de computação. Agora, ela passa a ser usada como ferramenta para falar sobre a **existência** ou **não existência** de algoritmos.

Ideia principal

Quando dizemos que um problema é decidível, queremos dizer que existe uma Máquina de Turing que resolve esse problema e sempre para.

Conexão com a Tese de Church-Turing

Se aceitamos que Máquina de Turing captura a noção informal de algoritmo, então provar que nenhuma MT decide um problema equivale a dizer que nenhum algoritmo geral resolve esse problema.

Problema, instância, entrada e saída

Antes de discutir decidibilidade, precisamos deixar claro o que significa resolver um problema.

Termo	Significado
Problema	Descrição geral de uma tarefa computacional.
Instância	Um caso específico daquele problema.
Entrada	Codificação da instância fornecida à máquina ou algoritmo.
Saída	Resposta produzida pela máquina.

Síntese

Um algoritmo não deve funcionar apenas para alguns exemplos. Ele deve resolver corretamente todas as instâncias válidas do problema.

Exemplo: aceitação por um AFD

Considere o problema:

Dado um AFD B e uma palavra w , B aceita w ?

- **Problema:** testar aceitação por um AFD.
- **Instância:** um AFD específico B e uma palavra específica w .
- **Entrada:** uma codificação $\langle B, w \rangle$.
- **Saída:** SIM, se B aceita w ; NÃO, caso contrário.

A notação $\langle B, w \rangle$ indica que os objetos matemáticos foram codificados de alguma maneira para serem manipulados por uma Máquina de Turing.

Problemas de decisão

Conceito

Um problema de decisão é um problema cuja resposta é sempre SIM ou NÃO.

Essa forma é central em teoria da computação porque permite transformar problemas em linguagens.

Exemplo

Problema: dado um grafo G e dois vértices s e t , existe caminho de s até t ?

Entrada: $\langle G, s, t \rangle$.

Saída: SIM ou NÃO.

Problemas de busca e otimização também podem ser relacionados a problemas de decisão, mas decidibilidade é normalmente apresentada primeiro nessa forma.

Linguagem como problema de decisão

Um problema de decisão pode ser representado como uma linguagem: o conjunto das codificações das instâncias cuja resposta é SIM.

$$L = \{w \mid w \text{ codifica uma instância positiva do problema}\}$$

Exemplo

Para o problema “o AFD B aceita a palavra w ?”, definimos a linguagem:

$$A_{DFA} = \{\langle B, w \rangle \mid B \text{ é um AFD que aceita } w\}.$$

Decidir o problema é equivalente a decidir se uma entrada pertence ou não a essa linguagem.

O que significa decidir uma linguagem?

Uma Máquina de Turing M **decide** uma linguagem L quando, para toda entrada w :

$$w \in L \Rightarrow M \text{ aceita } w$$

$$w \notin L \Rightarrow M \text{ rejeita } w$$

Além disso, a máquina deve parar nos dois casos.

Decidir = responder corretamente + sempre parar.

Se a máquina aceita os casos positivos, mas pode entrar em loop nos negativos, ela não é um decisor.

Exemplo rápido: linguagem dos múltiplos de 3

Problema de decisão:

Dado um número natural n , n é múltiplo de 3?

Linguagem correspondente:

$$MULT3 = \{ \langle n \rangle \mid n \text{ é múltiplo de } 3 \}$$

Exemplos:

$$\langle 6 \rangle, \langle 12 \rangle, \langle 0 \rangle \in MULT3$$

$$\langle 7 \rangle, \langle 10 \rangle \notin MULT3$$

Essa linguagem é decidível: basta dividir n por 3 e verificar o resto. O algoritmo termina para qualquer número natural finito representado na entrada.

Por que revisar a Hierarquia de Chomsky agora?

Antes de provar problemas de decidibilidade, vale localizar os principais modelos já estudados em uma mesma visão geral.

Ideia da revisão

A Hierarquia de Chomsky organiza classes de linguagens de acordo com as restrições das gramáticas e com o poder dos modelos capazes de reconhecê-las.

Regulares $\subset$ Livres de Contexto $\subset$ Sensíveis ao Contexto $\subset$ Recursivamente Enumeráveis

Síntese

Essa revisão ajuda a entender por que começamos com AFDs e GLCs, mas depois precisamos discutir Máquinas de Turing e limites da computação.

Hierarquia de Chomsky: visão geral

Tipo	Classe	Modelo associado	Exemplo breve
Tipo 3	Linguagens regulares	AFD, AFN, expressão regular	Palavras binárias que terminam em 1
Tipo 2	Linguagens livres de contexto	Autômato com Pilha; GLC	$\{a^n b^n \mid n \geq 0\}$
Tipo 1	Linguagens sensíveis ao contexto	Autômato linearmente limitado	$\{a^n b^n c^n \mid n \geq 1\}$
Tipo 0	Linguagens recursivamente enumeráveis	Máquina de Turing	A_{TM}

Mensagem principal

Quanto mais geral é a classe de linguagens, maior é o poder de memória e processamento exigido pelo modelo reconhecedor.

Tipo 3: linguagens regulares

Conceito

As linguagens regulares são as mais restritas da hierarquia. Elas podem ser reconhecidas por AFDs ou AFNs e podem ser descritas por expressões regulares.

Modelo computacional

Um AFD possui apenas uma quantidade finita de estados. Portanto, ele só consegue armazenar uma quantidade finita de informação sobre o prefixo já lido.

Exemplo

$$L = \{w \in \{0,1\}^* \mid w \text{ termina em } 1\}$$

Exemplos aceitos: 1, 01, 101, 0001. Para reconhecer essa linguagem, basta lembrar se o último símbolo lido foi 0 ou 1.

Tipo 2: linguagens livres de contexto

Conceito

As linguagens livres de contexto são reconhecidas por Autômatos com Pilha e descritas por Gramáticas Livres de Contexto.

Modelo computacional

A pilha fornece uma memória adicional. Ela permite guardar uma quantidade não limitada de símbolos, mas o acesso é restrito ao topo da pilha.

Exemplo

$$L = \{a^n b^n \mid n \geq 0\}$$

Exemplos aceitos: ε , ab, aabb, aaabbb. Um AP pode empilhar uma marca para cada a e desempilhar uma marca para cada b.

Tipo 1: linguagens sensíveis ao contexto

Conceito

As linguagens sensíveis ao contexto são mais gerais que as livres de contexto e exigem modelos com mais capacidade de memória e controle.

Modelo computacional

Um modelo associado é o Autômato Linearmente Limitado, que pode ser visto como uma Máquina de Turing cuja fita disponível é limitada proporcionalmente ao tamanho da entrada.

Exemplo

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

Exemplos aceitos: abc, aabbcc, aaabbbccc. Agora é necessário comparar três quantidades:
 $\#a = \#b = \#c$.

Tipo 0: linguagens recursivamente enumeráveis

Conceito

As linguagens recursivamente enumeráveis formam a classe mais geral da hierarquia. Elas são reconhecidas por Máquinas de Turing.

Ponto essencial

Uma Máquina de Turing pode aceitar, rejeitar ou executar para sempre. Portanto, reconhecer uma linguagem não significa necessariamente decidir essa linguagem.

Exemplo

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ é uma MT que aceita } w \}$$

Podemos reconhecer A_{TM} simulando M sobre w . Se M aceitar, aceitamos. Mas, se M entrar em loop, a simulação também pode não terminar.

Da Hierarquia de Chomsky para decidibilidade

Até aqui, a pergunta principal era:

Qual modelo consegue reconhecer uma linguagem?

Da Hierarquia de Chomsky para decidibilidade

Até aqui, a pergunta principal era:

Qual modelo consegue reconhecer uma linguagem?

A partir de agora, a pergunta muda:

Existe um algoritmo que sempre termina com uma resposta correta?

Conexão com as provas do capítulo

Vamos provar que alguns problemas sobre AFDs, AFNs, expressões regulares e GLCs são decidíveis. Depois veremos que problemas semelhantes sobre MTs podem deixar de ser decidíveis.

Síntese

Uma linguagem pode ser reconhecível por MT e, ainda assim, não ser decidível.

Antes da prova: por que começar com A_{DFA} ?

O primeiro teorema de decidibilidade do capítulo é sobre A_{DFA} porque ele estabelece o padrão básico que será repetido em outros problemas.

O que queremos mostrar?

Queremos mostrar que existe um algoritmo que recebe a descrição de um AFD e uma palavra, simula o autômato e sempre termina com uma resposta correta.

- O AFD é um modelo simples e sempre consome uma palavra finita em tempo finito.
- Isso permite introduzir a ideia de **decisor** sem ainda enfrentar loops infinitos.
- O resultado serve como base para problemas mais complexos, como A_{NFA} e A_{REG} .

Decidibilidade de A_{DFA}

O primeiro exemplo importante do capítulo é:

$$A_{DFA} = \{ \langle B, w \rangle \mid B \text{ é um AFD que aceita } w \}$$

Ideia do decisor

Receba a codificação de um AFD B e de uma palavra w . Simule o funcionamento de B sobre w . Se a simulação terminar em estado de aceitação, aceite; caso contrário, rejeite.

Isso funciona porque um AFD sempre lê uma palavra finita da esquerda para a direita e termina depois de processar o último símbolo.

Decisor para A_{DFA}

Um decisor para A_{DFA} pode ser descrito assim.

- 1 Na entrada $\langle B, w \rangle$, verifique se B é uma codificação válida de um AFD e se w é uma palavra sobre seu alfabeto.
- 2 Se a codificação for inválida, rejeite.
- 3 Inicialize o estado atual como o estado inicial de B .
- 4 Leia os símbolos de w , atualizando o estado atual conforme a função de transição de B .
- 5 Ao final da palavra, aceite se o estado atual for final; caso contrário, rejeite.

Síntese

A máquina sempre termina porque w é finita e cada símbolo é processado uma única vez.

Exemplo intuitivo de A_{DFA}

Suponha que B reconhece palavras binárias que terminam em 1.

Entrada do problema:

$$\langle B, 101 \rangle$$

Simulação de B :

$$q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0 \xrightarrow{1} q_1$$

Se q_1 é estado final, então:

$$\langle B, 101 \rangle \in A_{DFA}$$

Portanto, o decisor aceita essa entrada.

Antes da prova: por que estudar A_{NFA} ?

Depois de provar que A_{DFA} é decidível, o próximo passo é perguntar se a mesma ideia vale para AFNs.

Ponto pedagógico

A prova de A_{NFA} mostra uma técnica recorrente: resolver um problema novo convertendo-o para um problema já decidido.

- Um AFN pode ter vários caminhos possíveis de computação.
- Em vez de simular diretamente todos os caminhos, convertemos o AFN para um AFD equivalente.
- Depois reutilizamos o decisor de A_{DFA} .

Decidibilidade de A_{NFA}

Agora considere:

$$A_{NFA} = \{ \langle B, w \rangle \mid B \text{ é um AFN que aceita } w \}$$

Em vez de simular diretamente todas as escolhas não determinísticas, podemos usar um resultado já conhecido:

Todo AFN pode ser convertido em um AFD equivalente.

Estratégia:

- 1 Receber $\langle B, w \rangle$, em que B é um AFN.
- 2 Converter B para um AFD equivalente C .
- 3 Rodar o decisor de A_{DFA} sobre $\langle C, w \rangle$.

Por que essa prova é importante?

A prova de A_{NFA} mostra uma técnica recorrente na disciplina:

resolver um problema novo usando um problema já resolvido

Aqui, fazemos:

$$\langle B, w \rangle \longmapsto \langle C, w \rangle$$

onde C é um AFD equivalente ao AFN B .

Mensagem didática

Se conseguimos transformar uma instância de um problema em uma instância equivalente de outro problema decidível, obtemos um decisor para o problema original.

Antes da prova: por que estudar A_{REX} ?

A prova de A_{REX} reforça que decidibilidade não depende apenas da forma como a linguagem foi descrita.

Ideia central

Se uma expressão regular pode ser convertida para um autômato equivalente, então podemos transformar o problema em outro que já sabemos decidir.

- A entrada agora não é um autômato, mas uma expressão regular.
- Mesmo assim, expressões regulares, AFNs e AFDs descrevem exatamente as linguagens regulares.
- A prova destaca o papel das **conversões efetivas** entre modelos equivalentes.

Decidibilidade de A_{REX}

O capítulo também define:

$$A_{REX} = \{ \langle R, w \rangle \mid R \text{ é uma expressão regular que gera } w \}$$

A estratégia é encadear conversões conhecidas:

Expressão regular $\rightarrow$ AFN $\rightarrow$ AFD $\rightarrow$ simulação

- 1 Converta a expressão regular R em um AFN equivalente A .
- 2 Use o decisor de A_{NFA} em $\langle A, w \rangle$.
- 3 Aceite ou rejeite conforme o resultado.

Logo, A_{REX} é decidível.

Equivalência de representações regulares

Para linguagens regulares, temos três formas equivalentes de representação:

$$\text{expressões regulares} \equiv \text{AFNs} \equiv \text{AFDs}$$

Do ponto de vista de decidibilidade, isso é poderoso porque uma Máquina de Turing pode converter uma representação em outra.

Consequência

Se sabemos decidir um problema em uma dessas representações, muitas vezes conseguimos decidir o problema nas outras representações por conversão.

Essa ideia aparecerá novamente quando estudarmos reduções entre problemas.

Antes da prova: por que estudar E_{DFA} ?

Até aqui perguntamos se uma máquina aceita uma palavra específica. Agora mudamos a pergunta.

Nova pergunta

O AFD aceita **alguma** palavra, ou sua linguagem é vazia?

- Não podemos testar palavra por palavra, pois existem infinitas palavras possíveis.
- A estrutura finita do AFD permite outra estratégia: analisar alcançabilidade de estados.
- Esse teorema será usado depois como sub-rotina para provar a decidibilidade de EQ_{DFA} .

Decidibilidade de E_{DFA}

Agora o problema muda: em vez de perguntar se um AFD aceita uma palavra específica, perguntamos se ele aceita **alguma** palavra.

$$E_{DFA} = \{\langle A \rangle \mid A \text{ é um AFD e } L(A) = \emptyset\}$$

Pergunta: a linguagem do AFD A é vazia?

Ideia

Um AFD aceita alguma palavra se, e somente se, algum estado final é alcançável a partir do estado inicial.

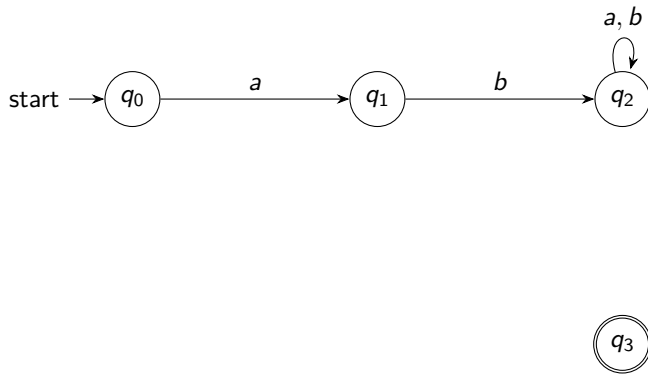
Decisor para E_{DFA}

O algoritmo para E_{DFA} é essencialmente um algoritmo de alcançabilidade no grafo de estados do AFD.

- 1 Receba $\langle A \rangle$, em que A é um AFD.
- 2 Marque o estado inicial de A .
- 3 Enquanto houver transições saindo de estados marcados para estados não marcados, marque esses novos estados.
- 4 Ao final, verifique se algum estado final foi marcado.
- 5 Se nenhum final foi marcado, aceite, pois $L(A) = \emptyset$.
- 6 Se algum final foi marcado, rejeite, pois $L(A) \neq \emptyset$.

Exemplo intuitivo de E_{DFA}

Considere um AFD cujo estado inicial é q_0 e cujo único estado final é q_3 .



Estados alcançáveis a partir de q_0 : q_0, q_1, q_2 . O estado final q_3 não é alcançável. Logo, nenhuma palavra leva o AFD a um estado final, e a linguagem é vazia.

Antes da prova: por que estudar EQ_{DFA} ?

O problema EQ_{DFA} pergunta se dois autômatos reconhecem exatamente a mesma linguagem.

Por que isso é importante?

Ele mostra que podemos decidir propriedades sobre **duas descrições formais**, não apenas sobre uma máquina isolada.

- A estratégia não é testar todas as palavras, pois existem infinitas entradas possíveis.
- Construimos um novo AFD que aceita apenas palavras em que os dois autômatos discordam.
- Depois usamos E_{DFA} : se a linguagem de discordância é vazia, os AFDs são equivalentes.

Decidibilidade de EQ_{DFA}

Outro problema importante é verificar se dois AFDs reconhecem a mesma linguagem.

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A \text{ e } B \text{ são AFDs e } L(A) = L(B) \}$$

Ideia da prova

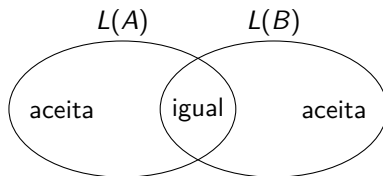
Construímos um AFD C que aceita exatamente as palavras em que A e B discordam. Se C não aceita nenhuma palavra, então A e B nunca discordam, portanto são equivalentes.

Diferença simétrica

A linguagem de discordância entre A e B é:

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

Ela contém palavras aceitas por exatamente um dos dois AFDs.



Se $L(C) = \emptyset$, não existe palavra que diferencie A e B .

Decisor para EQ_{DFA}

Decisor para EQ_{DFA} :

- 1 Receba $\langle A, B \rangle$, com dois AFDs.
- 2 Construa um AFD C para a diferença simétrica de $L(A)$ e $L(B)$.
- 3 Execute o decisor de E_{DFA} sobre $\langle C \rangle$.
- 4 Se E_{DFA} aceitar, então $L(C) = \emptyset$; aceite $\langle A, B \rangle$.
- 5 Se E_{DFA} rejeitar, então existe palavra que diferencia os AFDs; rejeite.

Síntese

A equivalência de AFDs é decidível porque linguagens regulares são fechadas por complemento, interseção e união, e porque vazio de AFD é decidível.

Antes da prova: por que estudar A_{CFG} ?

Agora saímos das linguagens regulares e entramos nas linguagens livres de contexto.

Pergunta

Dada uma gramática livre de contexto G e uma palavra w , existe uma derivação de G que gera w ?

- O problema é importante porque se relaciona à análise sintática de linguagens de programação.
- A dificuldade é que uma gramática pode possuir infinitas derivações.
- A prova mostra como transformar uma busca potencialmente infinita em uma busca finita.

Decidibilidade de A_{CFG}

Agora passamos para gramáticas livres de contexto.

$$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ é uma GLC que gera } w \}$$

Pergunta:

A gramática livre de contexto G gera a palavra w ?

Esse problema é importante porque se relaciona com o reconhecimento sintático de programas em linguagens de programação.

A primeira ideia falha

Uma ideia aparentemente natural seria:

- 1 Gere todas as derivações possíveis de G .
- 2 Verifique se alguma delas produz w .

Pergunta

Por que isso não é um decisor?

Porque pode haver infinitas derivações. Se G não gera w , a busca pode continuar para sempre.

Conclusão

Essa estratégia pode reconhecer casos positivos, mas não garante término nos casos negativos. Portanto, ainda não temos um decisor.

Forma Normal de Chomsky

Para transformar a ideia em um decisor, usamos a Forma Normal de Chomsky.

Ideia essencial

Se uma GLC está em Forma Normal de Chomsky, então qualquer derivação de uma palavra w , com $|w| = n > 0$, possui exatamente $2n - 1$ passos.

Assim, em vez de procurar infinitamente, podemos testar apenas derivações de tamanho limitado.

busca infinita $\longrightarrow$ busca finita

Decisor para A_{CFG}

Um decisor para A_{CFG} funciona assim.

- 1 Receba $\langle G, w \rangle$, em que G é uma GLC.
- 2 Converta G para uma gramática equivalente em Forma Normal de Chomsky.
- 3 Seja $n = |w|$. Liste todas as derivações com $2n - 1$ passos; para $n = 0$, trate o caso da palavra vazia separadamente.
- 4 Se alguma derivação gera w , aceite.
- 5 Caso contrário, rejeite.

Síntese

O algoritmo pode ser ineficiente, mas isso não importa aqui. Para decidibilidade, importa que ele sempre termina.

Antes da prova: por que estudar E_{CFG} ?

O problema E_{CFG} pergunta se uma gramática gera alguma palavra terminal.

Por que este teorema é útil?

Ele mostra que algumas propriedades globais de gramáticas ainda podem ser decididas sem enumerar todas as palavras da linguagem.

- Testar todas as palavras seria inviável, pois há infinitas possibilidades.
- O algoritmo analisa a própria estrutura da gramática.
- A ideia central é verificar se a variável inicial consegue gerar uma cadeia composta apenas por terminais.

Decidibilidade de E_{CFG}

O problema do vazio para GLCs é:

$$E_{CFG} = \{\langle G \rangle \mid G \text{ é uma GLC e } L(G) = \emptyset\}$$

A pergunta é:

A variável inicial de G consegue gerar alguma palavra formada apenas por terminais?

A solução não testa palavras uma por uma. Em vez disso, ela verifica quais variáveis são capazes de gerar cadeias terminais.

Algoritmo de marcação para E_{CFG}

Decisor para E_{CFG} :

- 1 Marque todos os símbolos terminais.
- 2 Repita enquanto for possível marcar novas variáveis:
 - ▶ marque uma variável A se existir uma produção

$$A \rightarrow U_1 U_2 \cdots U_k$$

em que todos os símbolos U_i já estão marcados.

- 3 Se a variável inicial não foi marcada, aceite, pois $L(G) = \emptyset$.
- 4 Se a variável inicial foi marcada, rejeite, pois $L(G) \neq \emptyset$.

Como há finitas variáveis, o processo de marcação termina.

Exemplo: marcação em uma GLC

Considere a gramática:

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow C$$

$$C \rightarrow b$$

Primeiro, marcamos os terminais a e b . Depois, marcamos A , pois $A \rightarrow a$; marcamos C , pois $C \rightarrow b$; marcamos B , pois $B \rightarrow C$; e finalmente marcamos S , pois $S \rightarrow AB$. Como S foi marcado, $L(G) \neq \emptyset$.

Antes da prova: por que concluir que toda LLC é decidível?

Após estudar A_{CFG} , podemos transformar o resultado em uma conclusão mais ampla.

Ideia

Se uma linguagem é livre de contexto, então existe uma gramática que a gera. Para decidir se w pertence à linguagem, basta decidir se essa gramática gera w .

- O teorema conecta gramáticas, linguagens e Máquinas de Turing.
- Ele posiciona as linguagens livres de contexto dentro da classe das linguagens decidíveis.
- Isso ajuda a construir a hierarquia: regulares $\subseteq$ livres de contexto $\subseteq$ decidíveis.

Toda linguagem livre de contexto é decidível

Se A é uma linguagem livre de contexto, então existe uma GLC G tal que:

$$L(G) = A$$

Para decidir se uma palavra w pertence a A , basta executar o decisor de A_{CFG} sobre:

$$\langle G, w \rangle$$

Se o decisor aceita, aceitamos w . Se rejeita, rejeitamos w .

Toda linguagem livre de contexto é decidível.

Antes do contraste: por que falar de EQ_{CFG} ?

Depois de vários exemplos decidíveis, é importante mostrar que nem todo problema sobre gramáticas livres de contexto é decidível.

Motivação

EQ_{CFG} parece análogo a EQ_{DFA} , mas a técnica usada para AFDs não se transfere diretamente para GLCs.

- Para AFDs, usamos complemento, interseção, união e teste de vazio.
- Para GLCs, faltam propriedades de fechamento necessárias para repetir a prova.
- Esse contraste prepara a transição para os problemas indecidíveis.

Mas nem todo problema sobre GLCs é decidível

É importante separar duas afirmações:

Verdadeiro

Toda linguagem livre de contexto é decidível.

Também verdadeiro

Nem todo problema sobre gramáticas livres de contexto é decidível.

Um exemplo citado no capítulo é:

$$EQ_{CFG} = \{ \langle G, H \rangle \mid L(G) = L(H) \}$$

Esse problema, a equivalência entre duas GLCs, é indecidível.

Por que EQ_{CFG} é mais difícil que EQ_{DFA} ?

Para AFDs, usamos diferença simétrica:

$$(L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

Isso funciona porque linguagens regulares são fechadas por complemento, interseção e união.

Para linguagens livres de contexto, a situação é diferente. Elas não possuem todas as propriedades de fechamento necessárias para repetir a mesma estratégia.

Síntese

A decidibilidade de um problema depende não apenas do objeto estudado, mas também das operações disponíveis sobre esse objeto.

Reconhecibilidade por Máquina de Turing

Conceito

Uma linguagem é Turing-reconhecível se existe uma Máquina de Turing que aceita todas as palavras que pertencem à linguagem.

A diferença está no comportamento para palavras que **não** pertencem à linguagem.

- Se $w \in L$, a máquina deve aceitar e parar.
- Se $w \notin L$, a máquina pode rejeitar, mas também pode executar para sempre.

Síntese

Reconhecer é mais fraco do que decidir, porque não exige parada em todas as entradas.

Decisor versus reconhecedor

Entrada	Decisor	Reconhecedor
$w \in L$	aceita e para	aceita e para
$w \notin L$	rejeita e para	rejeita ou não para
Sempre para?	sim	não necessariamente

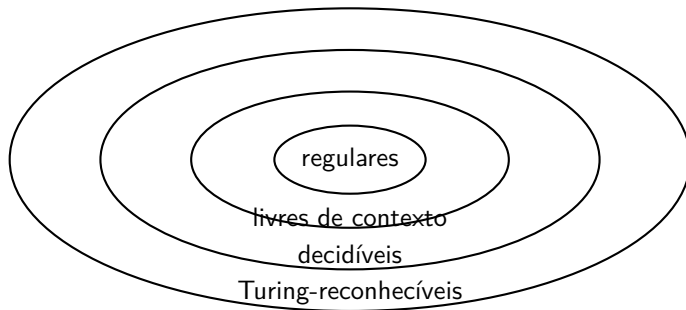
Consequência

Todo decisor também é um reconhecedor. Porém, nem todo reconhecedor é um decisor.

Essa distinção será central para entender A_{TM} .

Relação entre classes de linguagens

O capítulo organiza várias classes de linguagens em uma relação de inclusão.



$$\text{Regulares} \subseteq \text{LLC} \subseteq \text{Decidíveis} \subseteq \text{Turing-reconhecíveis}$$

Antes do argumento: por que contar máquinas e linguagens?

Antes de mostrar uma linguagem indecidível específica, o capítulo apresenta uma razão geral para existirem limites computacionais.

Ideia do argumento

Existem apenas contavelmente muitas Máquinas de Turing, mas existem incontavelmente muitas linguagens.

- Cada MT pode reconhecer no máximo uma linguagem.
- Se há mais linguagens do que máquinas, algumas linguagens ficam sem reconhecedor.
- Esse resultado mostra que a limitação não é acidental: ela é inevitável.

Por que existem linguagens não reconhecíveis?

A ideia do capítulo é comparar quantidades infinitas.

- Toda Máquina de Turing pode ser descrita por uma palavra finita $\langle M \rangle$.
- O conjunto de todas as palavras finitas sobre um alfabeto é contável.
- Portanto, o conjunto de todas as Máquinas de Turing é contável.

Assim, podemos imaginar uma lista:

$$M_1, M_2, M_3, \dots$$

com todas as Máquinas de Turing possíveis.

Existem mais linguagens que máquinas

Uma linguagem sobre Σ é qualquer subconjunto de Σ^* .

$$L \subseteq \Sigma^*$$

Como existem incontáveis subconjuntos possíveis de Σ^* , existem incontáveis linguagens.

Contraste

Há apenas contavelmente muitas Máquinas de Turing, mas incontavelmente muitas linguagens.

Logo, algumas linguagens não são reconhecidas por nenhuma MT.

Ideia intuitiva de diagonalização

A diagonalização é uma técnica para mostrar que uma lista não contém todos os objetos possíveis.

	1	2	3	4
x_1	0	1	1	0
x_2	1	1	0	0
x_3	0	0	0	1
x_4	1	1	1	1

Se construirmos uma nova sequência trocando cada elemento da diagonal, ela será diferente de x_1 na primeira posição, de x_2 na segunda, de x_3 na terceira, e assim por diante.

Diagonalização e indecidibilidade

A mesma ideia aparece em provas de indecidibilidade.

- Suponha que temos uma lista de todas as máquinas.
- Suponha que uma máquina especial consegue prever o comportamento de todas as outras.
- Construimos uma nova máquina que faz o oposto da previsão em uma entrada cuidadosamente escolhida.

Essa construção cria uma contradição: a máquina teria que se comportar de forma oposta ao próprio comportamento.

Síntese

A diagonalização é uma forma de construir um objeto que escapa de qualquer tentativa de listagem ou previsão completa.

Definição de A_{TM}

O capítulo define:

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ é uma MT e } M \text{ aceita } w\}$$

Essa linguagem representa o seguinte problema de decisão:

Dada uma Máquina de Turing M e uma palavra w , M aceita w ?

Ela é parecida com A_{DFA} , mas há uma diferença fundamental: uma Máquina de Turing pode executar para sempre.

Antes da prova: por que A_{TM} é diferente de A_{DFA} ?

A_{TM} é o análogo de A_{DFA} , mas substituindo o AFD por uma Máquina de Turing.

Pergunta

Dada uma MT M e uma palavra w , M aceita w ?

- Para AFDs, a simulação sempre termina.
- Para MTs, a simulação pode aceitar, rejeitar ou entrar em loop.
- Por isso, a simulação direta fornece um reconhecedor, mas não necessariamente um decisor.

A_{TM} é Turing-reconhecível

Existe uma Máquina de Turing universal U que reconhece A_{TM} .

Na entrada $\langle M, w \rangle$:

- 1 Simule M sobre w .
- 2 Se M entrar em estado de aceitação, aceite.
- 3 Se M rejeitar, rejeite.

Ponto importante

Se M entra em loop sobre w , a simulação também entra em loop. Portanto, esse procedimento reconhece A_{TM} , mas não decide A_{TM} .

Comparação entre A_{DFA} e A_{TM}

Problema	Simulação	Conclusão
A_{DFA}	AFD sempre termina após ler a palavra finita.	Decidível.
A_{TM}	MT pode aceitar, rejeitar ou entrar em loop.	Reconhecível, mas não decidível.

Síntese

A diferença não é a capacidade de simular. A diferença é a garantia de parada.

Antes da prova: por que provar que A_{TM} é indecidível?

A_{TM} é um dos primeiros exemplos concretos de uma linguagem Turing-reconhecível que não é decidível.

Por que este teorema é central?

Ele mostra que simular uma máquina universalmente não resolve o problema de decisão, pois a máquina simulada pode nunca parar.

- O objetivo é provar que não existe um decisor perfeito para aceitação por MT.
- A prova usa contradição e autorreferência.
- Esse resultado fundamenta a discussão posterior sobre o Problema da Parada.

Supondo que A_{TM} fosse decidível

A prova de indecidibilidade começa por contradição.

Suponha que existe uma Máquina de Turing H que decide A_{TM} . Então, para toda entrada $\langle M, w \rangle$:

$$H(\langle M, w \rangle) = \begin{cases} \text{aceita,} & \text{se } M \text{ aceita } w \\ \text{rejeita,} & \text{se } M \text{ não aceita } w \end{cases}$$

Como H seria um decisor, ela sempre terminaria.

Construindo a máquina D

Usando H como sub-rotina, construímos uma nova máquina D .

Na entrada $\langle M \rangle$, a máquina D :

- 1 executa H sobre $\langle M, \langle M \rangle \rangle$;
- 2 se H aceita, D rejeita;
- 3 se H rejeita, D aceita.

Assim:

$$D(\langle M \rangle) = \begin{cases} \text{aceita,} & \text{se } M \text{ não aceita } \langle M \rangle \\ \text{rejeita,} & \text{se } M \text{ aceita } \langle M \rangle \end{cases}$$

Executando D sobre ela mesma

Agora vem a autorreferência:

$$D(\langle D \rangle)$$

Substituindo M por D , temos:

$$D(\langle D \rangle) = \begin{cases} \text{aceita,} & \text{se } D \text{ não aceita } \langle D \rangle \\ \text{rejeita,} & \text{se } D \text{ aceita } \langle D \rangle \end{cases}$$

Isso é uma contradição. Se D aceita, deveria rejeitar. Se rejeita, deveria aceitar.

A_{TM} é indecidível.

Interpretação da prova

O mais importante não é decorar todos os símbolos da prova, mas entender sua estrutura.

- 1 Assumimos que existe um decisor perfeito H para A_{TM} .
- 2 Usamos H para construir uma máquina D que faz o oposto do previsto.
- 3 Executamos D sobre sua própria descrição.
- 4 Obtemos uma contradição.

Síntese

Logo, a suposição inicial estava errada: não existe decisor para A_{TM} .

Relação com o Problema da Parada

O Problema da Parada pergunta:

$$HALT = \{ \langle M, w \rangle \mid M \text{ para quando executada sobre } w \}$$

Essa pergunta é ligeiramente diferente de A_{TM} :

- A_{TM} : M aceita w ?
- $HALT$: M para sobre w , aceitando ou rejeitando?

Ambos expressam a mesma dificuldade fundamental: não existe algoritmo geral capaz de prever completamente o comportamento de qualquer programa sobre qualquer entrada.

O que a indecidibilidade não significa

A indecidibilidade não significa que nunca conseguimos analisar programas.

Exemplo

Um programa que executa apenas:

```
print("Olá")
```

claramente termina.

O que a teoria afirma é mais específico:

não existe um algoritmo geral que funcione para todos os programas e todas as entradas.

Isso explica por que ferramentas de análise podem ser úteis na prática, mesmo sem serem perfeitas em todos os casos possíveis.

Complemento e co-reconhecibilidade

O complemento de uma linguagem A é:

$$\overline{A} = \{w \mid w \notin A\}$$

Dizemos que A é **co-Turing-reconhecível** quando seu complemento é Turing-reconhecível.

$$A \text{ é co-reconhecível} \iff \overline{A} \text{ é reconhecível}$$

Esse conceito permite caracterizar decidibilidade em termos de reconhecimento dos casos positivos e negativos.

Antes do teorema: por que relacionar reconhecibilidade e complemento?

Depois de separar decidibilidade e reconhecibilidade, precisamos entender quando um reconhecedor pode ser transformado em decisor.

Ideia principal

Se conseguimos reconhecer os casos positivos e também reconhecer os casos negativos, então conseguimos decidir o problema executando os dois reconhecedores em paralelo.

- Um reconhecedor sozinho pode não parar nos casos negativos.
- Um reconhecedor do complemento aceita exatamente os casos negativos.
- Juntos, eles garantem que alguma das execuções terminará.

Teorema: reconhecível dos dois lados

O capítulo apresenta o seguinte resultado:

A é decidível $\iff A$ é Turing-reconhecível e $\overline{A}$ é Turing-reconhecível

Ideia

Se temos um reconhecedor para A e outro para $\overline{A}$, podemos rodar os dois em paralelo. Como toda palavra está em A ou em $\overline{A}$, um dos dois reconhecedores aceitará em algum momento.

Execução em paralelo

Dado w , suponha que:

- M_1 reconhece A ;
- M_2 reconhece $\bar{A}$.

Construímos um decisor M :

- 1 Execute um passo de $M_1(w)$.
- 2 Execute um passo de $M_2(w)$.
- 3 Repita alternando as duas simulações.
- 4 Se M_1 aceitar, aceite.
- 5 Se M_2 aceitar, rejeite.

Como w pertence a exatamente um dos dois conjuntos, uma das máquinas aceitará.

Consequência para $\overline{A_{TM}}$

Sabemos que:

A_{TM} é Turing-reconhecível

mas:

A_{TM} não é decidível

Se $\overline{A_{TM}}$ também fosse Turing-reconhecível, então A_{TM} seria decidível pelo teorema anterior. Isso contradiz a indecidibilidade de A_{TM} . Portanto:

$\overline{A_{TM}}$ não é Turing-reconhecível.

Mapa dos principais problemas do capítulo

Problema	Pergunta	Situação
A_{DFA}	AFD aceita uma palavra?	decidível
A_{NFA}	AFN aceita uma palavra?	decidível
A_{REG}	ER gera uma palavra?	decidível
E_{DFA}	Linguagem do AFD é vazia?	decidível
EQ_{DFA}	Dois AFDs são equivalentes?	decidível
A_{CFG}	GLC gera uma palavra?	decidível
E_{CFG}	Linguagem da GLC é vazia?	decidível
A_{TM}	MT aceita uma palavra?	reconhecível, indecidível

Resumo conceitual

- Um problema de decisão pode ser visto como uma linguagem.
- Uma linguagem é decidível quando existe uma MT que sempre para e responde corretamente.
- Uma linguagem é reconhecível quando existe uma MT que aceita as palavras pertencentes à linguagem, mas pode não parar nas demais.
- Muitos problemas sobre AFDs e GLCs são decidíveis.
- A_{TM} é reconhecível, mas não decidível.
- O complemento de A_{TM} não é reconhecível.

Síntese

A grande mensagem é que a computação possui limites formais.

Exercício guiado 1: enunciado

Considere a linguagem:

$$A_{PAR} = \{\langle B \rangle \mid B \text{ é um AFD que aceita alguma palavra de comprimento par}\}$$

Mostre, em alto nível, que A_{PAR} é decidível.

Orientação para resolver em sala

A pergunta não é se uma palavra específica é aceita. A pergunta é se existe pelo menos uma palavra de comprimento par aceita por B .

Pergunta

Como podemos transformar esse problema em um teste de linguagem vazia?

Exercício guiado 1: decomposição da ideia

Queremos saber se:

$$L(B) \cap PAR \neq \emptyset$$

onde:

$$PAR = \{w \mid |w| \text{ é par}\}$$

A linguagem PAR é regular. Portanto, podemos construir um AFD P que alterna entre dois estados:

- estado par: número de símbolos lidos é par;
- estado ímpar: número de símbolos lidos é ímpar.

Depois, construímos um AFD C para:

$$L(C) = L(B) \cap L(P)$$

Exercício guiado 1: solução passo a passo

Decisor para A_{PAR} :

- 1 Receba $\langle B \rangle$, em que B é um AFD.
- 2 Construa um AFD P que reconhece palavras de comprimento par.
- 3 Construa o produto de B com P , obtendo um AFD C tal que:

$$L(C) = L(B) \cap L(P)$$

- 4 Execute o decisor de E_{DFA} em $\langle C \rangle$.
- 5 Se E_{DFA} aceitar, então $L(C) = \emptyset$; rejeite $\langle B \rangle$.
- 6 Se E_{DFA} rejeitar, então $L(C) \neq \emptyset$; aceite $\langle B \rangle$.

Exercício guiado 1: interpretação

Se $L(C) \neq \emptyset$, então existe alguma palavra w tal que:

$$w \in L(B)$$

e:

$$|w| \text{ é par}$$

Logo, B aceita alguma palavra de comprimento par.

Se $L(C) = \emptyset$, nenhuma palavra aceita por B possui comprimento par.

Síntese

O problema é decidível porque foi reduzido a operações regulares e ao decisor de E_{DFA} .

Exercício guiado 2: enunciado

Use o algoritmo de marcação para decidir se a linguagem da gramática abaixo é vazia.

$$S \rightarrow AB \mid D$$

$$A \rightarrow a$$

$$B \rightarrow C$$

$$C \rightarrow b$$

$$D \rightarrow DE$$

$$E \rightarrow e$$

Pergunta

A variável inicial S consegue gerar alguma palavra formada apenas por terminais?



Exercício guiado 2: passo 1

Primeiro, marcamos os terminais:

a, b, e

Agora procuramos variáveis que possuem produção formada apenas por símbolos já marcados.

- $A \rightarrow a$: como a está marcado, marcamos A .
- $C \rightarrow b$: como b está marcado, marcamos C .
- $E \rightarrow e$: como e está marcado, marcamos E .

Exercício guiado 2: passo 2

Agora temos marcados:

a, b, e, A, C, E

Verificamos novamente as produções.

- $B \rightarrow C$: como C está marcado, marcamos B .
- $S \rightarrow AB$: como A e B estão marcados, marcamos S .
- $D \rightarrow DE$: mesmo que E esteja marcado, D aparece do lado direito e ainda não foi marcado; essa regra não permite marcar D inicialmente.

Como S foi marcado, a variável inicial gera alguma cadeia terminal.

Exercício guiado 2: conclusão

Como S foi marcado, concluimos que:

$$L(G) \neq \emptyset$$

Na verdade, podemos exibir uma palavra gerada:

$$S \Rightarrow AB \Rightarrow aB \Rightarrow aC \Rightarrow ab$$

Portanto, a gramática gera pelo menos a palavra ab .

Resposta do decisor de E_{CFG}

Como E_{CFG} contém gramáticas com linguagem vazia, o decisor de E_{CFG} deve **rejeitar** $\langle G \rangle$.

Exercício guiado 3: enunciado

Considere o seguinte procedimento para A_{TM} :

- 1 Na entrada $\langle M, w \rangle$, simule M sobre w .
- 2 Se M aceitar, aceite.
- 3 Se M rejeitar, rejeite.

Explique:

- 1 por que esse procedimento reconhece A_{TM} ;
- 2 por que ele não decide A_{TM} ;
- 3 qual entrada evidencia o problema.

Exercício guiado 3: casos possíveis

Ao simular $M(w)$, três coisas podem acontecer.

Comportamento de $M(w)$	Comportamento do simulador
M aceita w	simulador aceita
M rejeita w	simulador rejeita
M entra em loop sobre w	simulador também entra em loop

Nos dois primeiros casos, o procedimento dá uma resposta. No terceiro caso, ele não para.

Exercício guiado 3: conclusão

O procedimento reconhece A_{TM} porque, se $\langle M, w \rangle \in A_{TM}$, então M aceita w , e a simulação eventualmente chega ao estado de aceitação.

Mas ele não decide A_{TM} porque, se M entra em loop sobre w , a simulação nunca termina. Um decisor precisaria rejeitar nesses casos.

reconhecedor: aceita os casos positivos

decisor: aceita positivos e rejeita negativos, sempre parando

Síntese

Esse é o ponto central: A_{TM} é reconhecível, mas não decidível.

Fechamento da aula

A aula começou com a pergunta sobre os limites dos algoritmos e chegou à seguinte conclusão:

Nem todo problema computacional possui um algoritmo decisor.

- Alguns problemas são decidíveis, como vários problemas sobre AFDs e GLCs.
- Alguns problemas são reconhecíveis, mas não decidíveis, como A_{TM} .
- Alguns problemas nem sequer são Turing-reconhecíveis, como $\overline{A_{TM}}$.

Na próxima etapa, podemos perguntar como comparar problemas por meio de reduções.